USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Engineering Documentation

cyd-os

DOCUMENT	REVISION	STATUS	CLASSIFICATION
cyd-os	—	—	Internal

Purpose of this set

These reports record the design of cyd-os in enough detail that the project can be picked up cold — by someone else, or by the author after six months away — without re-deriving decisions from the source.

Each report states **what was decided, what it was measured against, and what remains unverified**. Claims that were tested on hardware are marked as such. Claims taken from documentation or reasoning are marked separately, because on a from-scratch kernel the difference between "this is true" and "this should be true" is the difference between a working boot and a silent reboot.

Index

ID	Title	Covers
UM-CYDOS-001	System Architecture and Scope	Layer model, what is built vs borrowed, the isolation problem, why a bytecode VM
UM-CYDOS-002	Boot Chain and Image Format	ROM → 2nd stage → kernel, image header, measured boot trace
UM-CYDOS-003	Xtensa ABI Selection	Windowed vs call0, codegen evidence, consequences for the scheduler
UM-CYDOS-004	Memory Map and Allocation Policy	ESP32 regions, our layout, the bootloader overlap risk
UM-CYDOS-005	Build and Flash Pipeline	Toolchain, flags and why each one, reproducing a build, why not PlatformIO
UM-CYDOS-006	Milestone 0 Verification Report	Test method, captured output, pass/fail per assertion
UM-CYDOS-007	Development Roadmap M1–M5	Each milestone, its risks, and its exit criteria
UM-CYDOS-008	Milestone 1 Verification Report	Vectors, timer source and level choice, interrupt entry/exit, measured results
UM-CYDOS-009	Milestone 2 Verification Report	Task model, context frame, scheduler, the zero-overhead `LOOP` defect, watchdog correction
UM-CYDOS-010	Milestone 3 Verification Report	Heap allocator, arena model and bounds checking, and the measured DRAM budget
UM-CYDOS-011	Flash Cache and Read-Only Data Placement	Mapping `.rodata` into flash, the 0x20 page congruence, and the cache-off hazard
UM-CYDOS-012	Milestone 4 Verification Report	Register-based bytecode VM, the ISA, the assembler, and containment of malformed programs

Reading order

New to the project: **001** → **002** → **004** → **003**. That gives the shape of the system, how it starts, where things live, and why the calling convention is unusual.

Picking up implementation work: **006** (what is known-good) then **007** (what is next and what it will break).

Reproducing a build: **005** alone is sufficient.

Status summary as of this revision

Item	State
Milestone 0 — kernel boots, self-checks pass	**Complete, verified on hardware**
Milestone 1 — timer interrupt, tick counter	**Complete, verified on hardware**
Milestone 2 — native task switching	**Complete, verified on hardware** — 3,400+ switches, zero corruption
Milestone 3 — heap and arena model	**Complete, verified on hardware** — all three exit criteria
Milestone 4 — bytecode interpreter	**Complete, verified on hardware** — program runs, six fault classes contained
Isolation	**Live** — every VM memory access bounds-checked, UM-CYDOS-012 §6.2
Full stack	**Running** — bytecode hosted in a preemptible native task, 3.3M instructions across 450 preemptions with zero accounting drift, UM-CYDOS-012 §6.6
DRAM budget	**Measured** — 167,680 B allocatable; a full framebuffer is unnecessary, UM-CYDOS-010 §7.2
Flash cache	**Enabled** — `.rodata` mapped from flash, UM-CYDOS-011
Version control	**Initialised 2026-08-14**
JTAG debug probe	Ordered, not in hand
Bootloader IRAM overlap	**Closed** — investigated and disproved, UM-CYDOS-004 §5
Panic handler	**Closed** — exercised deliberately with an `ill` instruction; prints EXCCAUSE/EPC and halts
Watchdogs	**Closed** — measured armed, now disabled and read back, UM-CYDOS-009 §8

> **Standing rule for interrupt handlers — clear `PS.EXCM` before calling C.** Hardware sets `EXCM` on interrupt entry, and while it is set the Xtensa zero-overhead loop-back is disabled: a hardware loop body executes once and falls through to whatever sits at `LEND`. GCC emits these loops for ordinary counted C loops, so this silently degrades **every C function reachable from a handler** — drivers and the VM interpreter included, not just the scheduler where it was found. `_handler_level3` now clears it; any future handler must do the same. UM-CYDOS-009 §6.